

Technical Requirements Document (TRD)

Project Name: Talent Intelligence Platform (Talent OS) Document Version: 1.0

1. Executive Summary

Traditional Applicant Tracking Systems (ATS) act purely as data repositories. The **Talent Intelligence Platform** is designed to sit on top of ATS data to dynamically score, predict, and visualize candidate potential. By combining robust data engineering, machine learning predictions (like Market Value), and a premium "Apple/Vercel-inspired" enterprise UI, it allows HR professionals to make immediate, data-driven hiring decisions.

2. System Architecture

The application follows a monolithic architecture built entirely in Python, utilizing Streamlit for the presentation layer. The architecture is split into three core layers:

1. **Data Ingestion & Transformation Layer:** Processes raw nested JSON candidate data into cleaned, relational tabular data (CSVs).
 2. **Machine Learning & Analytics Engine:** Generates predictive features, aggregate talent scores, and market value benchmarks.
 3. **Presentation Layer (Dashboard):** A highly customized Streamlit frontend that injects raw HTML/CSS to achieve a bespoke, premium SaaS aesthetic.
-

3. Technology Stack

- **Core Language:** Python 3.8 +
 - **Frontend Framework:** Streamlit
 - **Data Processing:** Pandas, NumPy
 - **Machine Learning:** Scikit-Learn (Random Forest algorithms)
 - **Data Visualization:** Plotly (Interactive Gauges, Donut Charts)
 - **Styling:** Custom HTML5 and Vanilla CSS (Injected via Streamlit)
-

4. Core Modules & Pipelines

4.1 Data Processing Pipeline

- **candidates_df.py:** The extraction script. It parses the nested `sample_candidates.json` file and splits it into multiple distinct, relational DataFrames (Candidates, Career, Skills, Education, Languages, Certifications, Assessments, Signals).
- **data_cleaning.py:** Normalizes text strings (e.g., standardizing skill names), handles missing NaN values, and ensures datatype consistency.
- **feature_engineering.py:** Derives advanced metrics from raw data, such as `total_skills`, `years_of_experience`, and aggregate `talent_score`.

- **f_merge.py**: The consolidation script that stitches all relational tables back into a unified `master_df.csv` using the `candidate_id` primary key.

4.2 Machine Learning & Analytics Engine

- **market_value_predictor.py**: Uses a Random Forest Regressor trained on experience, skill counts, and assessment scores to predict a candidate's fair market value.
- **hidden_talent_engine.py**: An algorithmic model designed to flag candidates with low traditional pedigree (e.g., non-tier-1 schools) but high empirical output (e.g., Github activity, assessment scores).
- **learning_velocity.py & candidate_potential.py**: Analytics models that track the speed at which candidates acquire new skills over their career timeline.

4.3 Frontend Dashboard (app.py)

The primary interface module. It loads the processed `output/` CSVs and renders the UI.

- **Dashboard Hub**: Displays high-level KPIs, animated metric gauges, and a Top Candidate Leaderboard.
 - **Candidate 360° Profile**: A detailed view of an individual candidate, structured using a "Bento Box" UI. It includes deep dives into their Experience, Skills, and Language Proficiencies.
 - **AI Copilot**: A predictive interface that pits a candidate's `expected_salary` against their ML-predicted `market_value` using a minimal visual progress track.
-

5. UI/UX Design System Specifications

The application overrides Streamlit's default components to achieve a "**Premium SaaS Data Sheet**" aesthetic.

- **Design Philosophy**: Minimalist, highly structured, and enterprise-grade.
 - **Typography**: Relies heavily on modern, crisp typography (e.g., Inter font family) with strict visual hierarchies (uppercase subheadings, tight letter-spacing).
 - **Containers**: Uses clean white cards with 1px hairlines (`#e2e8f0`) and exceptionally subtle drop shadows to create depth without clunkiness.
 - **Vibrant Components**: For data that scales (like Language Proficiency), the UI utilizes vibrant, Vercel-inspired continuous gradient progress bars (`#3b82f6` to `#6366f1`) to create an immediate "wow factor" and break up the tabular data.
-

6. Data Schema Overview

The pipeline ultimately produces `master_df`, which acts as the source of truth for the dashboard. **Primary Key:** `candidate_id` **Key Features Generated:**

- `expected_salary`
- `avg_assessment_score`
- `github_activity_score`
- `profile_completeness_score`
- `talent_score` (Composite Metric)

7. Future Deployment Considerations

- **Scalability:** The current file-based JSON ingestion should be migrated to a relational database (e.g., PostgreSQL) or Document Store (e.g., MongoDB) as the candidate pool scales.
- **State Management:** Dashboard currently re-runs models sequentially. Future iterations should cache ML predictions using `@st.cache_data` or migrate inference to a dedicated backend API endpoint (e.g., FastAPI).